

tracebox

Engineering
Real WebRTC Voice-
Agent Testing

Browser Automation • LangGraph • WebRTC • Realtime Observability • Session Evidence

Voice-agent testing • browser + media systems

1–100

bots per run

Supported batch size

8

LangGraph states

Explicit lifecycle

120s

max listen window

RMS silence detection

1

worker concurrency

Cloud Run execution

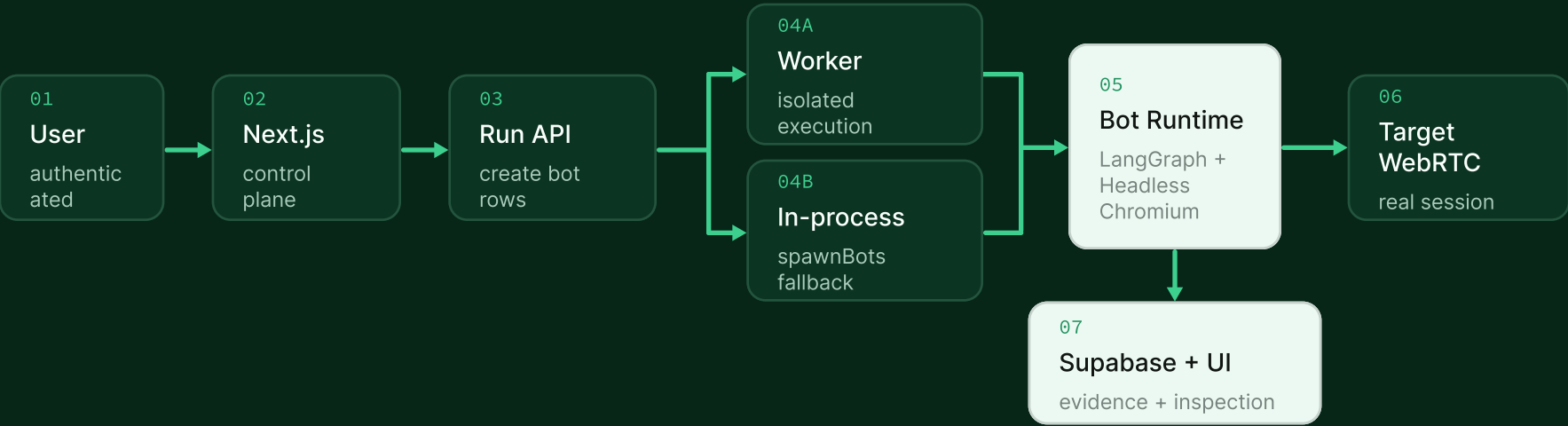
ENGINEERING NARRATIVE

Control plane → isolated browser execution → real WebRTC conversations → persistent evidence.

Built the browser-based testing workflow, eight-state LangGraph agent, WebRTC capture and injection, isolated worker execution, realtime inspection, and Supabase-backed run evidence.

SYSTEM OVERVIEW

High-level run dispatch from authenticated control plane into one of two execution modes.



ENGINEERING SCOPE

Control plane

Browser workers

LangGraph

WebRTC media

Realtime UI

Evidence capture

CORE TECHNOLOGIES

Next.js • TypeScript • LangGraph • Stagehand • Playwright • Chromium • Chrome DevTools Protocol

WebRTC • Deepgram Nova-2 • GPT-4.1 / Gemini • ElevenLabs • Supabase • FFmpeg • Cloud Run

Control Plane, Execution Plane & Ownership

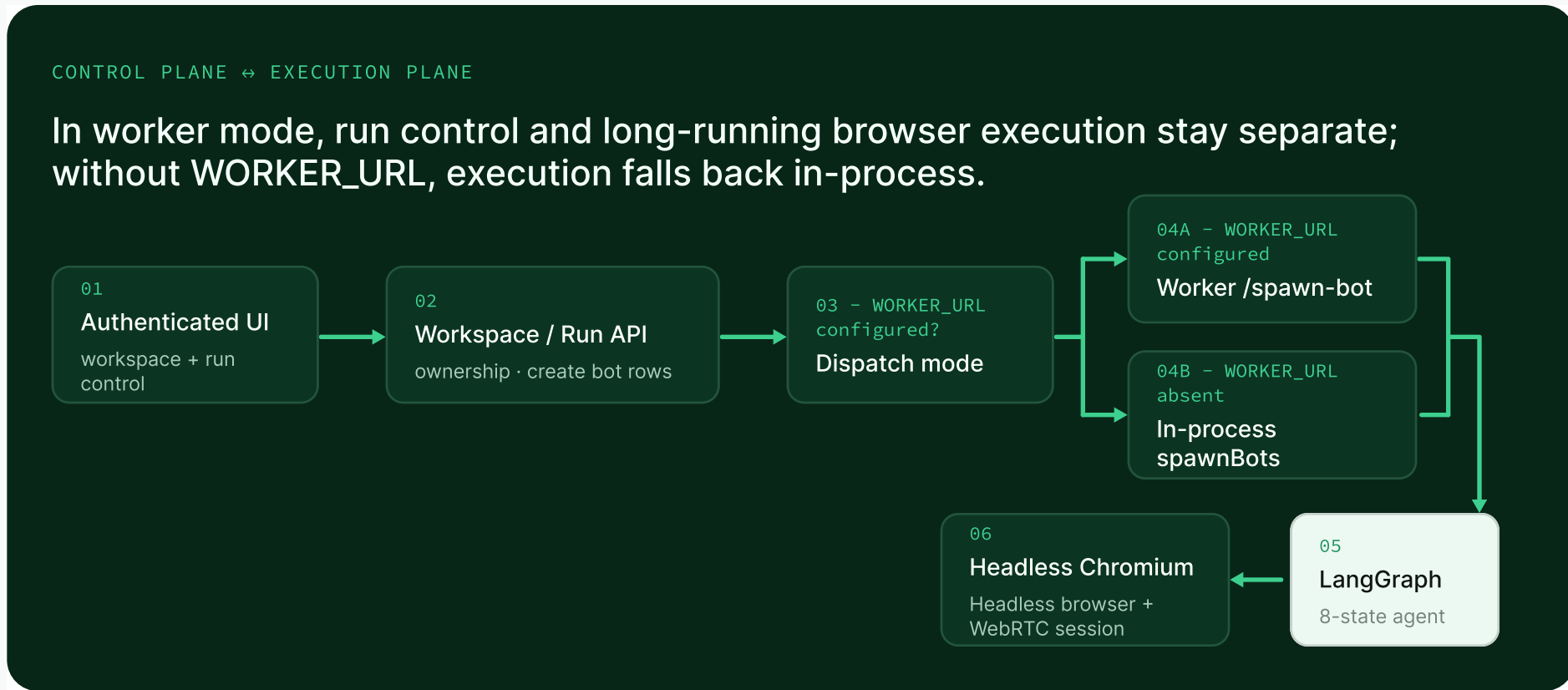
How runs are created, dispatched, executed, observed, and persisted.

ENGINEERING OWNERSHIP

Built the voice-agent testing workflow end to end

Eight-state LangGraph agent · browser/WebRTC control · isolated workers

· realtime inspection · persistence · session evidence.



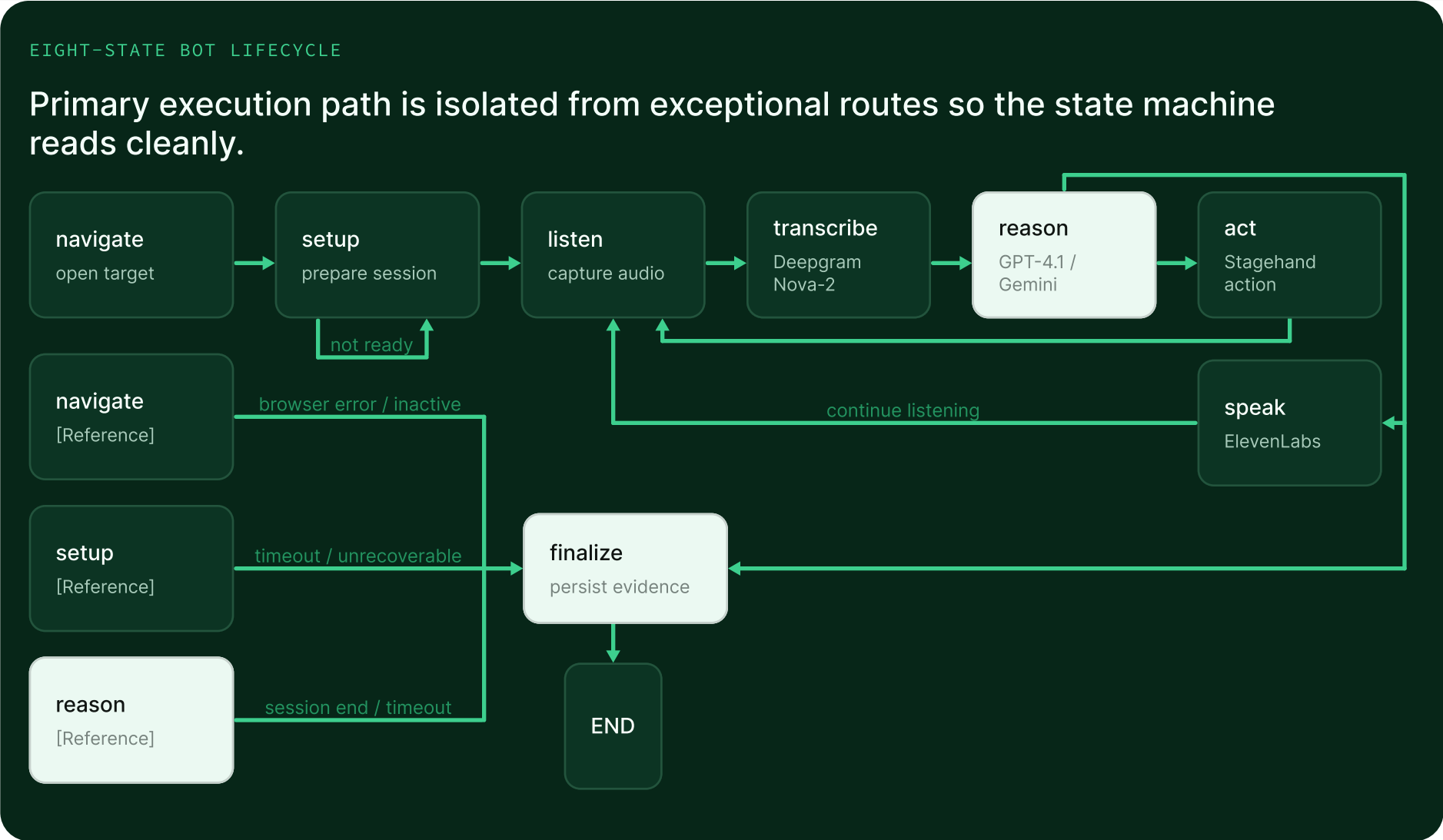
OWNERSHIP LEDGER

BUILT	Browser testing workflow browser-based voice-agent testing	DESIGNED + BUILT	Eight-state LangGraph agent navigate → finalize lifecycle
BUILT	WebRTC capture + injection real live browser conversations	ENGINEERED	RMS voice activity detection 120-second max listen window
BUILT	Isolated worker execution Cloud Run concurrency 1	BUILT	Local execution fallback in-process spawnBots path
BUILT	Realtime live inspection status + transcript + screencast	BUILT	Persistent session evidence screenshots · recording · trace · logs

Public engineering story: control/execution split, isolated headless browser workers, LangGraph lifecycle, real WebRTC media, realtime observability, evidence capture, deployment topology, and known limitations.

LangGraph Agent & Runtime Guardrails

An explicit eight-state lifecycle with retries, time limits, recovery, and finalization.

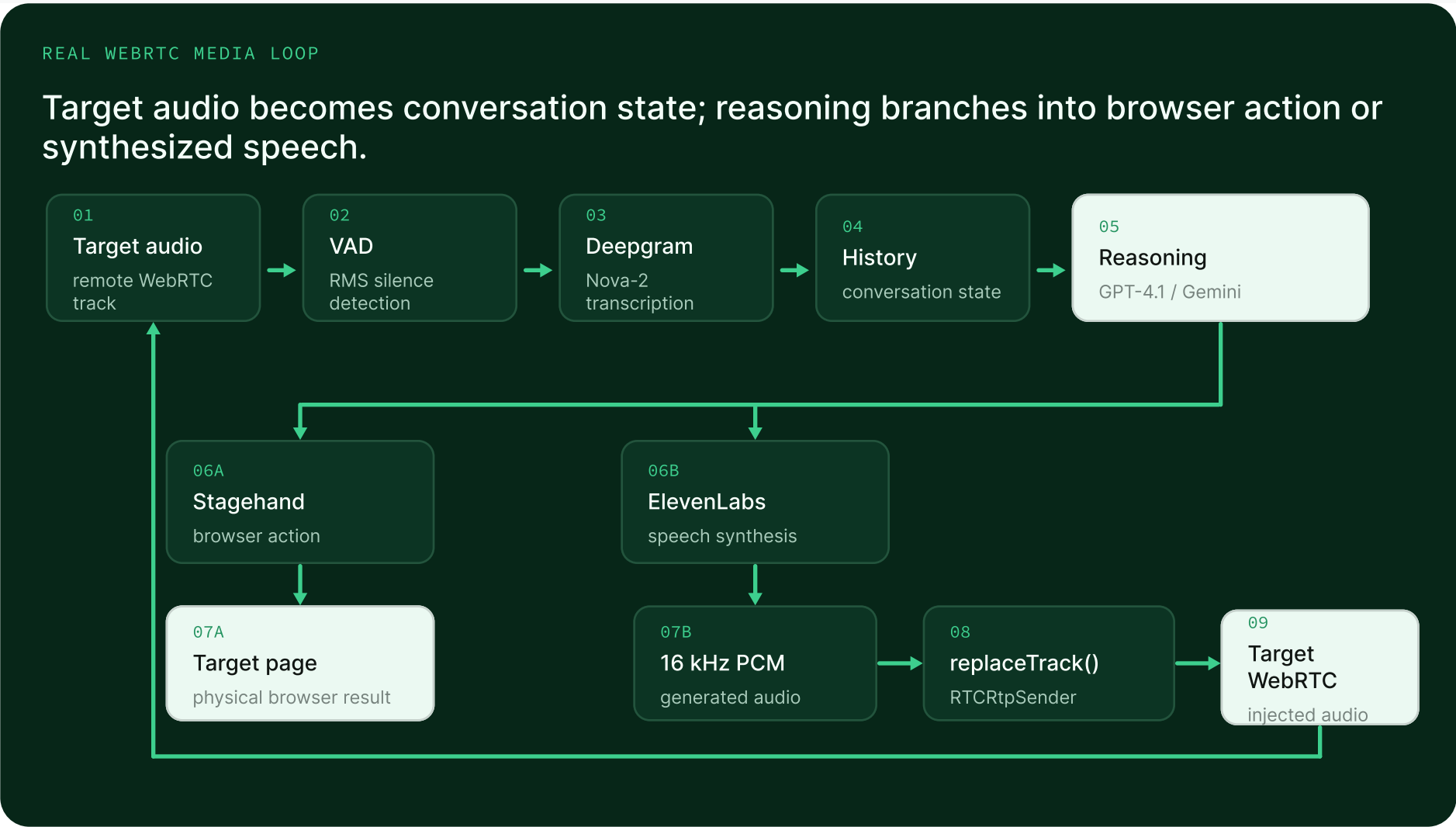


RUNTIME GUARDRAILS

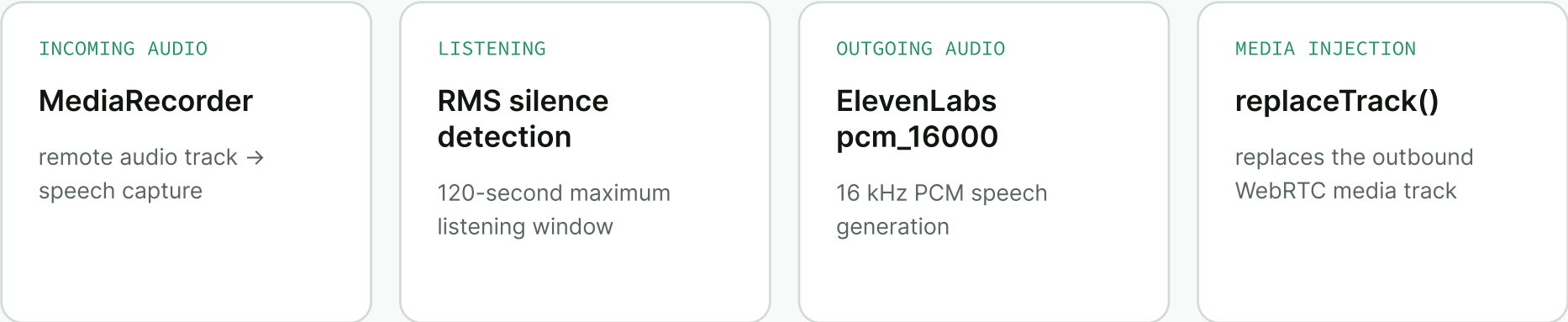
SESSION DEADLINE max_session_duration checked in setup, listen, and reason	SETUP CAP 50 turns setup stops after the configured cap	SILENT SESSION 20 turns reasoning stops without system transcript
STUCK RECOVERY 3 bot turns fresh screenshot before selecting next action	LISTEN WINDOW 120 seconds RMS-based silence / voice activity detection	STOP BEHAVIOR workspace + bots marks non-terminal bots stopped; listen checks status

Real-Time WebRTC Voice Loop

Capture live speech, reason over conversation and page state, then inject synthesized audio back into the active browser session.



WEBRTC MEDIA BRIDGE



SCREENSHOT-ASSISTED REASONING

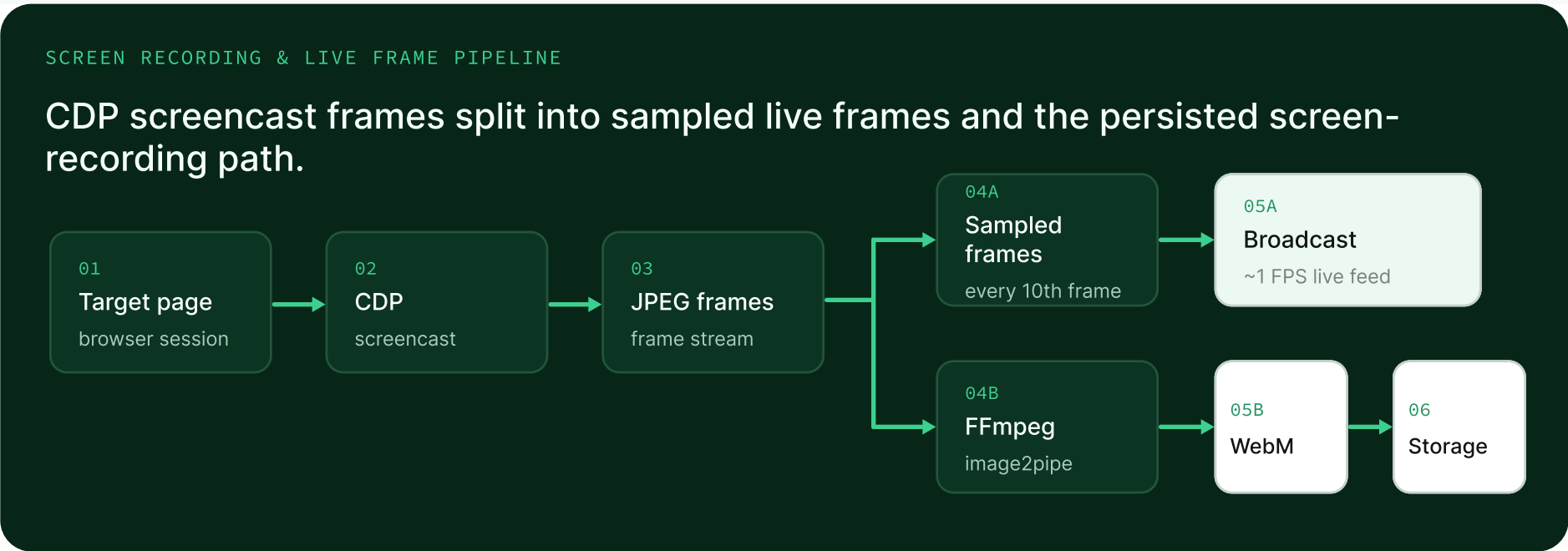
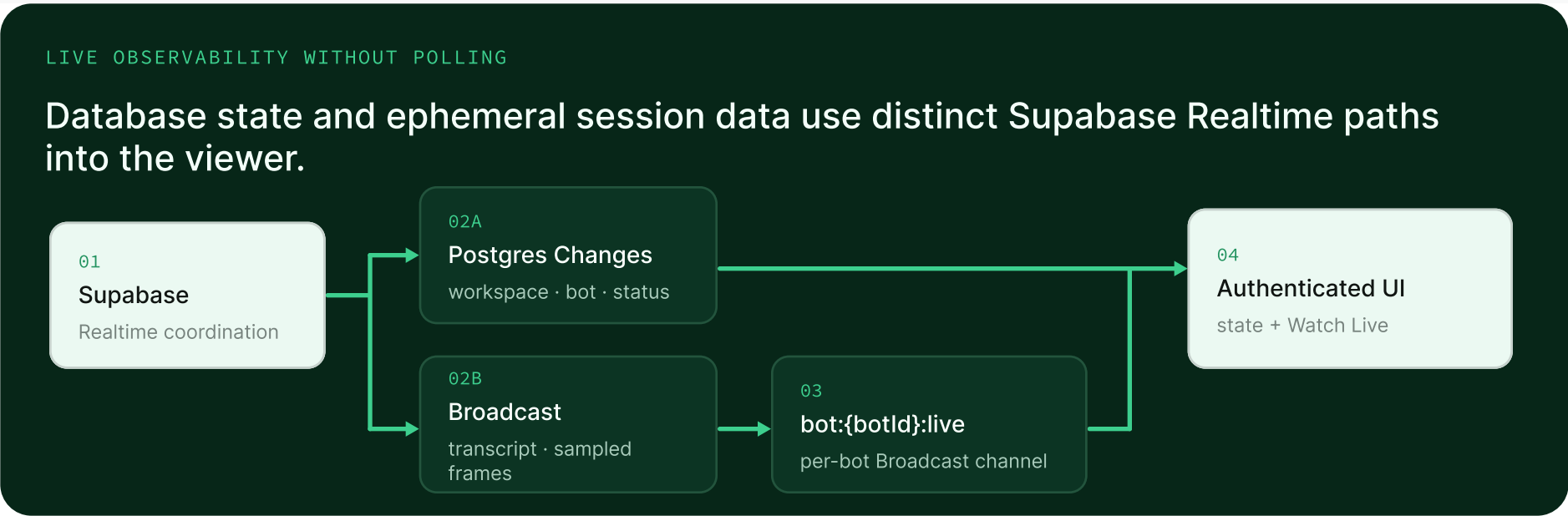
LATEST PAGE CONTEXT

Screenshots are captured after navigation and browser actions.

Each screenshot is uploaded for later inspection and kept as the latest raw PNG in graph state so the reasoning call can receive current page context directly.

Realtime Observability & Session Evidence

Live inspection without polling, plus recordings, screenshots, transcripts, timings, logs, and errors after a run.



EVIDENCE PER BOT

RECORDING WebM / audio recording URL and path	TRANSCRIPT JSON + normalized speaker + text + elapsed timestamp	SCREENSHOTS Storage URLs captured after navigation / actions
TRACE LangGraph timings per-node execution durations	LOGS per-bot server logs execution-side evidence	ERRORS error details terminal failure context

Deployment, Technical Facts & Limitations

What is confirmed by the implementation—and what the public case study deliberately does not claim.

TWO-SERVICE CLOUD RUN PATH

deploy.sh deploys the worker first, obtains its URL, then deploys the frontend with WORKER_URL.

01

Worker image

worker/Dockerfile

→

02

Cloud Run worker

concurrency 1

→

03

WORKER_URL

obtain + configure

→

04

Cloud Run frontend

frontend Dockerfile · uses worker URL

deploy.sh defaults to asia-south1 · GCP, Supabase, and provider configuration are provisioned separately · cloudbuild.yaml is an alternate combined-image path

CONFIRMED TECHNICAL FACTS

FACT	CONFIRMED VALUE	FRAMING
Bots per run	1–100	Configured supported batch size
LangGraph lifecycle	8 states	named lifecycle from navigate through finalize
Production worker model	concurrency 1	worker service configured with concurrency 1
Listening window	120 seconds	maximum RMS-based VAD listening window

KNOWN LIMITATIONS & BOUNDARY

SECURITY LIMITATION

Public-readable media buckets

recordings + screenshots are not private by default

TESTING LIMITATION

No automated test suite

no unit / integration / E2E / load-test files

RECORDING FALLBACK

Preserve available evidence

muxed → screen-only → audio-only

CLAIM BOUNDARY

No unsupported performance claims

no uptime / benchmark / SLA / load claims

Publication boundary · high-level engineering architecture and confirmed configuration only; no unsupported user, revenue, uptime, benchmark, reliability, privacy, or load-performance claims.